

Software System Development (SSD)

Monsoon 2026

https://serc-iiith.github.io/CS6302_M26.html

What is a System?

- Commonly used/understood definition
 - Set of inter-related components working together to achieve a common objective
- A system may be “**natural**” or “**Engineered**”
 - Solar System (Natural)
 - Telephone network system, power plants etc. (Engineered)
 - Systems have boundaries – due to various reasons

Exploratory programming

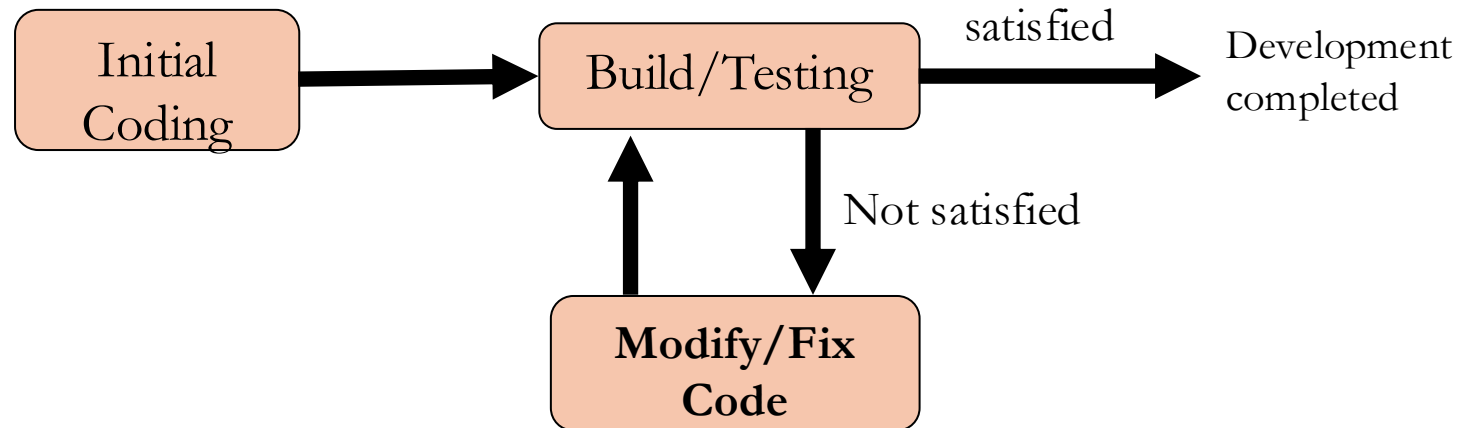
A bridge needs steel, permits and a survey before anyone builds. Software needs none of that.

So how do we actually build one? Almost everyone starts the same way !
write something that runs, then fix what breaks.

That habit has a name, and a cost.



**Exploratory
programming**

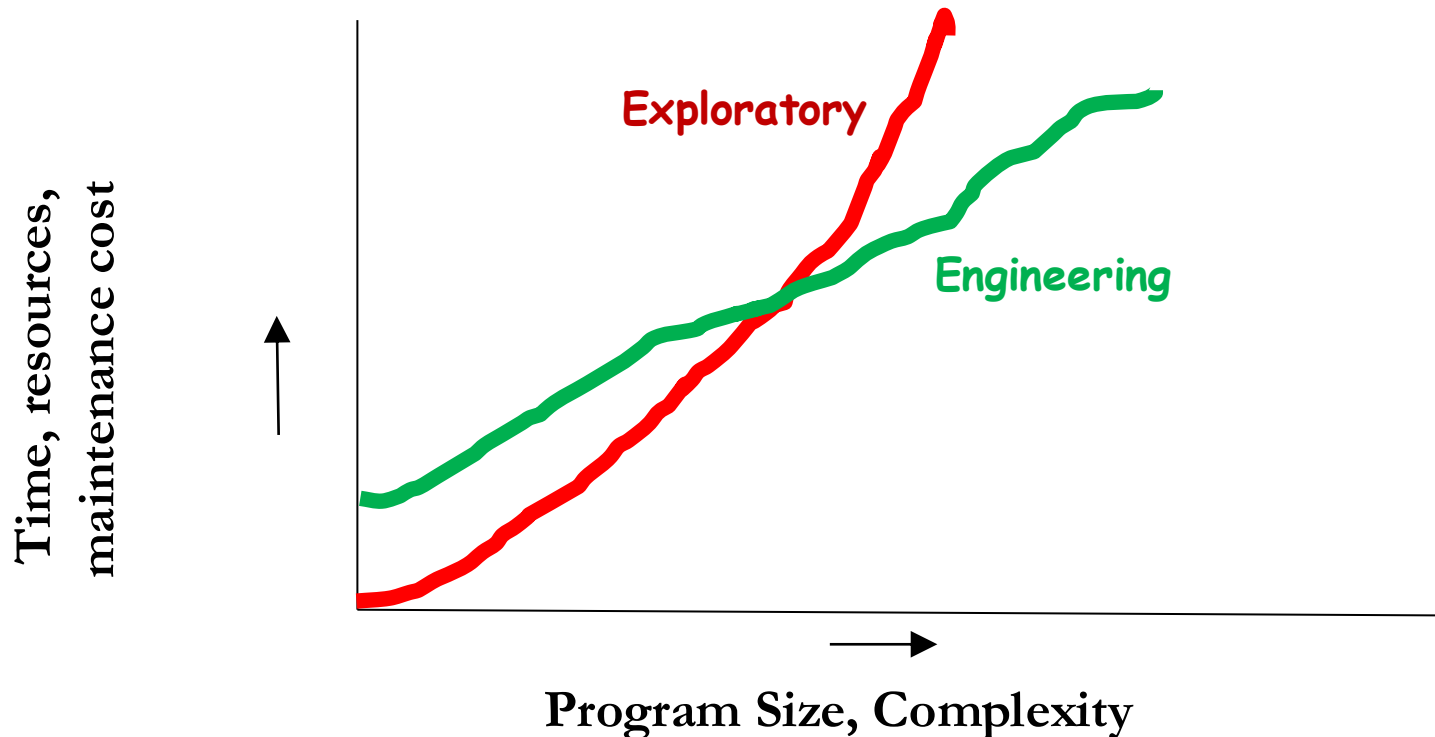


Will Exploratory programming work here ?



What is Wrong with the Exploratory Style?

- Can successfully be used for very small programs only
- Large programs can become unmaintainable
- Time and Effort required to develop a product can grow exponentially with program size



What do these have in common?



Large systems with different levels of abstractions.
Design needs to be thought about at different levels – from small modules up to the entire system

Design

Systems are complex...

Modern systems need a *lot* of software to operate.

How much?

- Even simple (modern) games have between 1 and 2 MLOC (million lines of code)
- Mahindra XEV 9S has 75 MLOC on-board.

$$1 \text{ LOC/min/SE} * 60 \text{ min/hr} * 40 \text{ hrs/wk} = 2,400 \text{ SLOC/wk/SE}$$

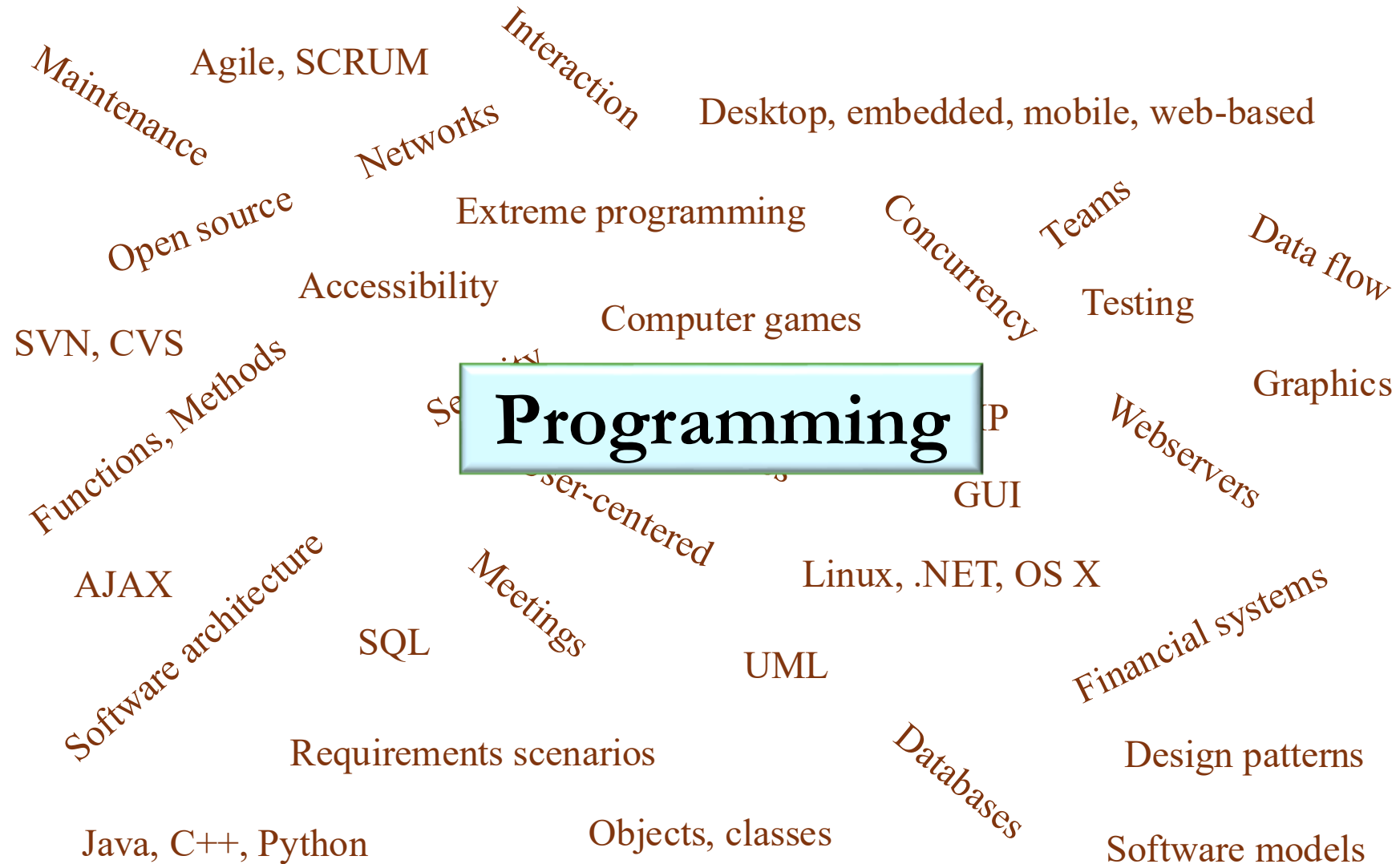
$$2,400 \text{ SLOC/wk/SE} * 50 \text{ wk/year} = 120,000 \text{ LOC/year/SE}$$

$$75 * 10^6 \text{ LOC} / 1.2 * 10^5 \text{ LOC/yr/SE} = \sim 625 \text{ SEs for the year}$$



Teamwork

(Software) engineers **get their hands dirty writing programs** using the latest technologies and techniques.



The software engineer's daily job is to answer questions about the software system...

- How can I help the customer? What is required to solve the customer's problem?
- How will the user interact with the system?
- What operating system, language, hardware is going to be used?
- What is the overall software system structure and how do different components interact with each other?
- What code do I have to write?
- How do I organize my team so we are effective?
- Can we finish the game in time to have it on the shelves for Christmas shopping?

To answer those questions, the software engineer must work with many people.

- Customers asking for the system
- People who will use the system
- Domain experts: banking, avionics, security, medical, scientists, ...
- Engineers from other engineering disciplines
- Most closely with the other software engineers on the project

Communication

SOFTWARE DEPLOYMENT FAILURE

KNIGHT CAPITAL GROUP - AUGUST 1, 2012

ROOT CAUSE:

Faulty software deployment.
Configuration error in trading application.
Poor release management.



IMPACT:

US\$440 MILLION LOSS
in 45 minutes.
Company required
emergency financing & sale.



A critical example of software process and release management failures.

Electronic Trading Firm

Complex!!!

One small upgrade. Easy???

Process

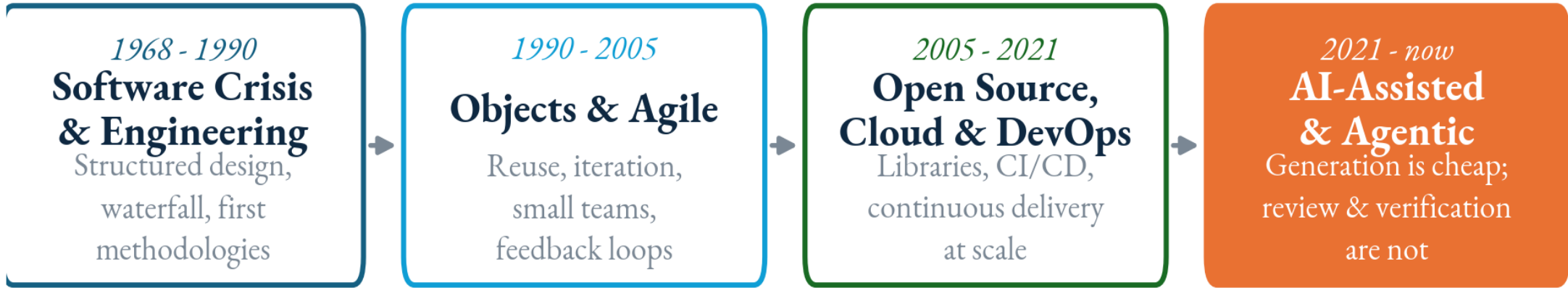
Estimated millions in lost revenue.

Ended up in selling the company

This is a financial-critical system.

The team needed a better understanding of the configuration and release process for developing a financial-critical system.

Software Development in the AI Era



- The 1968 NATO conference named the “**software crisis**”. Projects are late, over budget, unreliable. Fifty-eight years later, that still holds true...
- Every era made producing software cheaper. **No era has made engineering it easier.**

AI Changed the Cost of Code, Not the Cost of Correctness



- **Typing code was never the bottleneck.** Deciding what to build, and proving it works, always was.
- The cost of establishing that it is the **right** solution is unchanged. It simply arrives faster, and there is more of it.
- Adoption is near-universal. However developers distrusting AI accuracy rose from 31% to 46% in a single year.
- **Model Evaluation and Threat Research randomised trial (2025):** experienced open-source developers were **19% slower** with AI tools.

What AI does well today

**Writing
code**

Eliciting what is actually needed

Architecture & design trade-offs

Integration across teams and systems

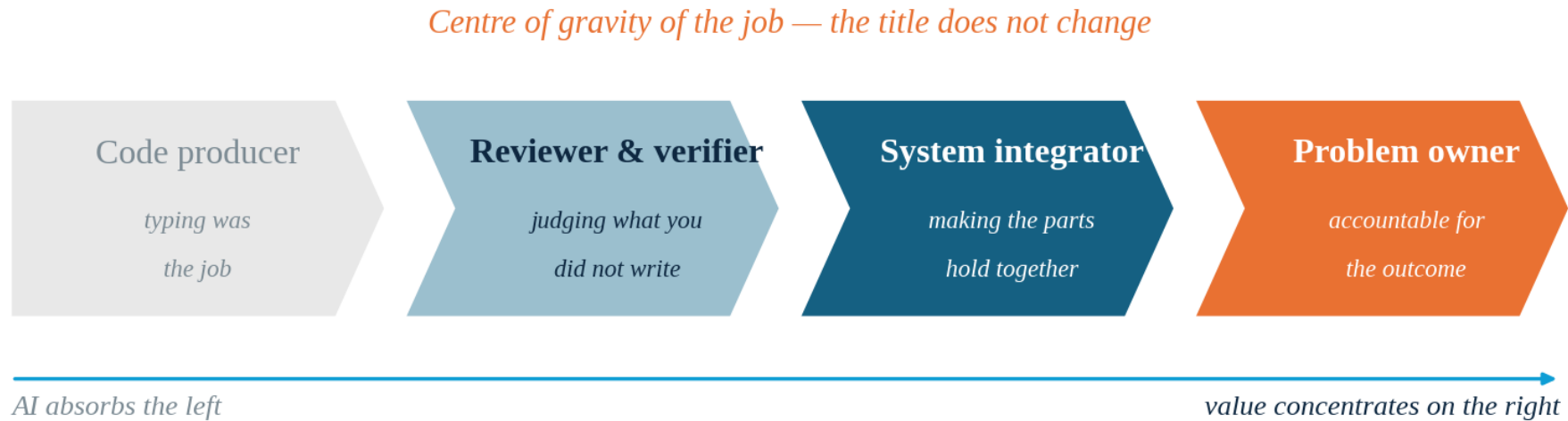
Verification, testing, safety

Operating & evolving it for years

Communication and accountability

Everything below the line is still yours.

Software Engineering Role Is Changing, Not Disappearing



- **Writing → judging:** The scarce skill is not producing code. It is deciding whether code you did not write is correct and safe.
- **Module → system:** Individual output is cheap. Owning how the pieces fit and evolve over years is not.
- **Backroom → frontline:** When a prototype takes hours, the expensive mistake is building the wrong thing quickly.

What This Means For You in This Course

- **Use AI (openly):** You can't avoid it. You will most likely be working with it for the rest of your careers.
- **Declare it:** Every assignment/activity/project asks what you used AI for and what you verified yourself. No declaration can cost you everything !!!
- **Own it:** You will be assessed on the judgement behind the code. Why this design, what breaks it, how you know it works.

Course Instructors



Dr. Sai Anirudh Karre

Data Product Manager, Phenom Inc.

Guest Faculty

Software Engineering Research Centre (SERC)

IIIT Hyderabad

Dr. Abhishek Singh

Assistant Professor,

Software Engineering Research Centre (SERC)

IIIT Hyderabad



Course Details

- The overall goal of this course is to provide a working knowledge of tools to build software systems
- **Course Structure:** 2 Classes per week (1 hour 25 min per class), Lab Work – 1 every week (3 hrs per week), Tutorial- 1 every week (as and when required)
- **Course Notes:** Reference Material and relevant notes will be made available on Moodle. Students are expected to read the notes, put on effort, work towards raising your problem-solving skills and learn things by doing.
- **Grading Criteria:** Assignments – 20 %, Quiz – 20%, MID Exam (Hackathon) – 20%, Lab/class Activities – 15 %, FINAL Exam (Hackathon) – 25 %
- **Facilitators:** Instructors and Teaching Assistants.
- **Time:** Honor Time and Come with learning mindset. Ensure that you record your queries and discuss them offline when we run out of time.
- **Lab Work:** Shell, HTML, CSS, JS, React, Flask, Python, SQL, NoSQL

Academic Honesty

- A helps B in task X
 - B doesn't get opportunity to do task X
 - B doesn't learn the skill to do task X
 - B gets spoiled, dependent and unfit for jobs requiring skills of X
 - You may think it is okay to do it only once and not repeat it. But when a thing is done once, it gets wired into the brain as being “okay”; and unless there is a strong reason, it will repeat.
- If you want to help, help to learn.